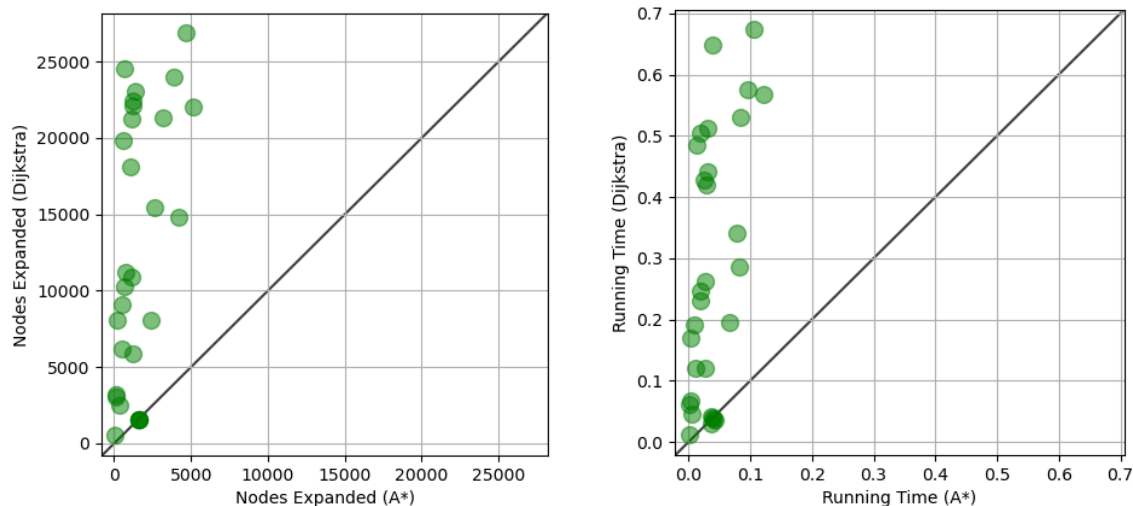


Question 1.

- a. For both graphs, each point represents the number of nodes expanded or running time of the dijkstra and heuristic search algorithms on the same problem.

Fig. 1.1 Scatter Plots of Nodes Expanded and Running Time between Dijkstra and A* Search



In the nodes_expanded chart, for small problems, both algorithms cover nearly as many nodes as the others (<5000) since the points are close to the diagonal reference line; however, as the problem grows, dijkstra search starts taking more and more nodes (up to more than 25000) while the number of nodes expanded by heuristic search are maintained under 5000 for the majority of the problems.

The same applies to the running_time graph, as the running times are close to each other for small problems (<0.05), but the gap grows as the problem becomes bigger, until the running time of the dijkstra search algorithm reaches 0.35 when most of the running times of the heuristic search are maintained under 0.05.

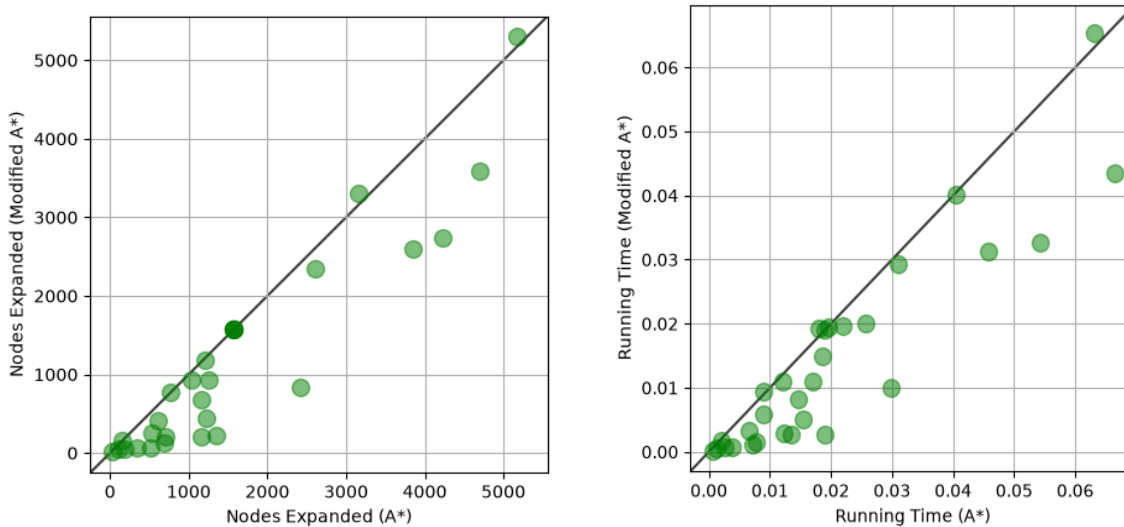
Overall, the graphs summarize that for path seeking problems, as the problems get larger, heuristic search is generally more efficient in terms of the number of nodes covered and time usage than dijkstra search.

- b. The distributions of points in both graphs are nearly identical to each other. Which is due to the fact that heuristic search is generally more efficient in terms of the number of nodes covered and time usage than dijkstra search.

Question 2.

I modified the code in main.py so that it now plots node expanded and running time between the original A* and modified A* (heuristic * 2) search:

Fig. 1.2 Scatter Plots of Nodes Expanded and Running Time between original A* and modified A* Search



While both algorithms are able to generate an optimal route from start to end node, their number of expanded nodes and amount of time differ

In the Node Expanded scatter plot, it is observed that most points fall slightly or moderately below the diagonal referencing line, indicating that under the same grid search problem, modified A* search expands moderately fewer nodes than the original A* search, although for some problems, the points are on the referencing line, meaning that two search algorithms expanded exactly the same number of nodes.

In the Running Time scatter plot, it is observed that most points fall slightly or moderately below the diagonal referencing line, indicating that under the same grid search problem, modified A* search uses moderately less time than the original A* search, although for some problems, the points are on the referencing line, meaning that two search algorithms used the same amount of time.

Question 3.

To modify the algorithm “so that the search does not update the cost and parent pointer to a node n once it finds a better path to n .”, I also add an open list check when deciding whether each child from the successor function should be pushed to the frontier. Specifically, the modified algorithm denies the child if the node already exists in the open list, even if the new child is more optimal.

By running `main.py`, for both search algorithms, it is observed that although the solution paths still connect start and goal nodes, for several problems, the encountered costs are greater than the expected costs, meaning that under these problems, the solution paths are not optimal.